Assignment 4.3

Vishnu Bommisetty

2026-04-14

Busiest Airports Table

Here I will be making a professional looking table for the Busiest Airports. This will ultimately show improvement from the previous table in the 3.4 assignment

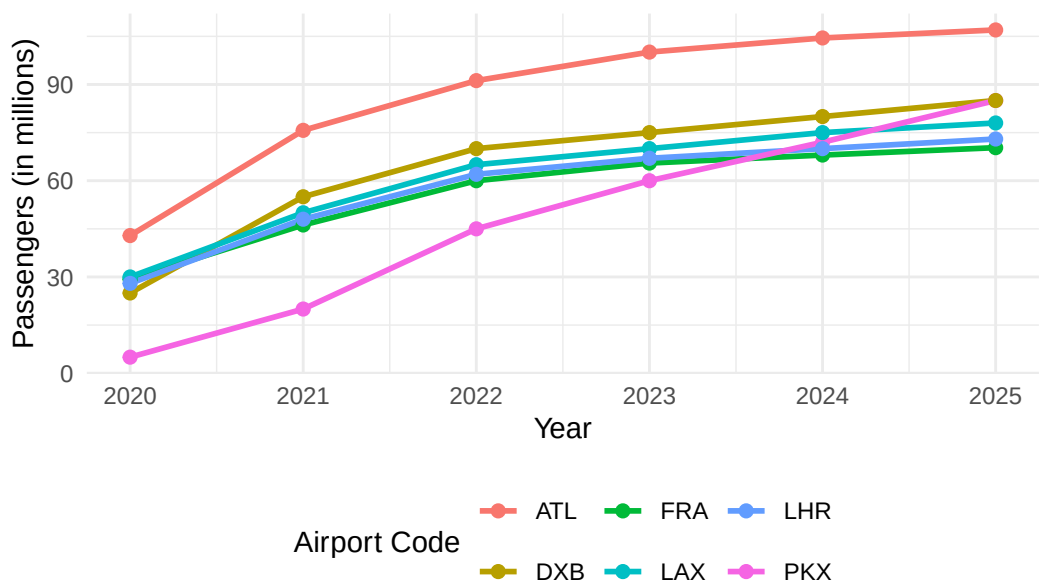
```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union


Attaching package: 'kableExtra'

The following object is masked from 'package:dplyr':

    group_rows

  Airport Year Passengers
1     ATL 2020       42.9
2     ATL 2021       75.7
3     ATL 2022       91.2
4     ATL 2023      100.1
5     ATL 2024      104.5
6     ATL 2025      107.0
```

Table 1: Table 1. Passenger Traffic (2020–2025) for Six Major Airports

Airport	2020	2021	2022	2023	2024	2025
ATL	42.9	75.7	91.2	100.1	104.5	107.0
DXB	25.0	55.0	70.0	75.0	80.0	85.0
FRA	29.3	46.2	60.0	65.5	68.0	70.3
LAX	30.0	50.0	65.0	70.0	75.0	78.0
LHR	28.0	48.0	62.0	67.0	70.0	73.0
PKX	5.0	20.0	45.0	60.0	72.0	85.0

```
  Airport Year Passengers
1     ATL 2020       42.9
2     ATL 2021       75.7
3     ATL 2022       91.2
4     ATL 2023      100.1
5     ATL 2024      104.5
6     ATL 2025      107.0
```

gure 1: Passenger Traffic Trends for Six of the Busiest Airports (2020-



What is being displayed with respect to the context:

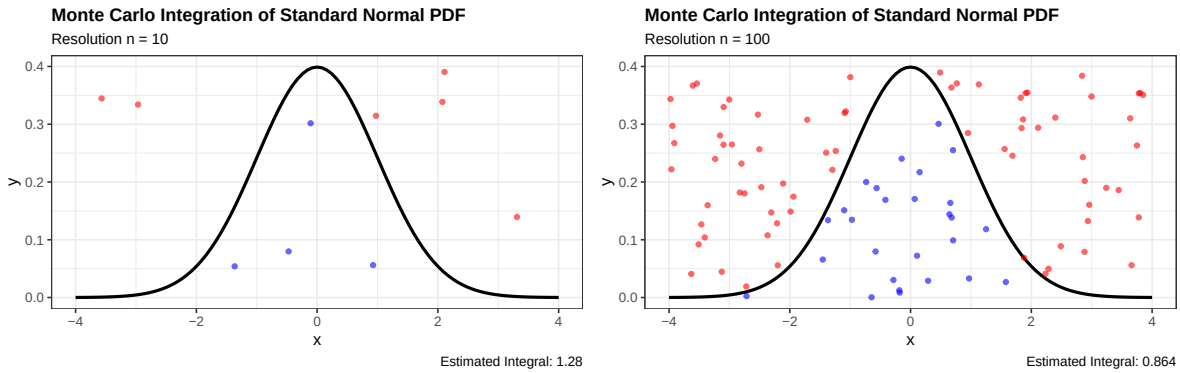
The table and plot both show the different trends and helps us understand how the world had a big drop in air travel during the pandemic and how it recovered in the coming years. The

table helps us see and compare the precise passenger counts each year, while the plot helps us see the overall recovery of each airport. One specific trend was how Atlanta was always dominant and far in front of the others every single year. Atlanta had the highest curve, and the table confirmed that it grew steadily.

Monte Carlo Integration Plots

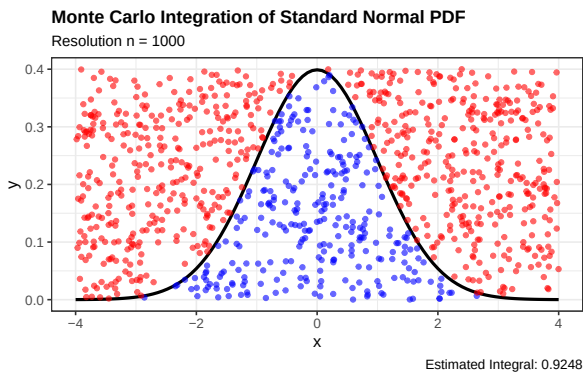
Here are the plots:

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.1     v readr     2.1.6
v lubridate 1.9.5     v stringr   1.6.0
v purrr     1.2.1     v tibble    3.3.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter()          masks stats::filter()
x kableExtra::group_rows() masks dplyr::group_rows()
x dplyr::lag()             masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

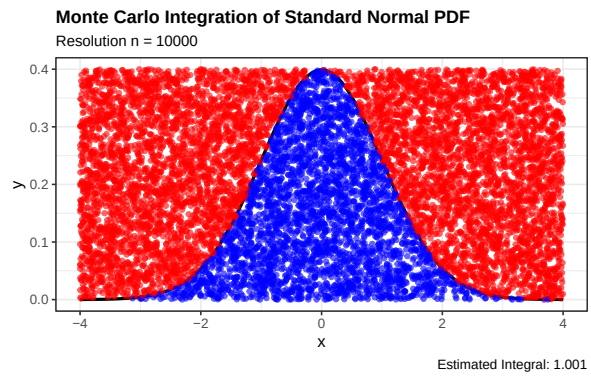


(a) n = 10

(b) n = 100



(c) n = 1,000



(d) n = 10,000

Figure 1: Monte Carlo Numeric Integration Plots

In Figure 1, you can see how the estimate behavews as the number of simulated points increases. When there is 10, the points are sparse with the estimate being unstable. When there is 100 and 1000 points, we can see in the plots that the points are beginning to become more evenly spaced and the curve is much more normal and recognizable.

Plan Informed-Response

My Plan:

Firstly, the goal is to understand how the data is structured and see that there is one woman per row. The data is not tidy as the time is spread across the columns and we need one row per woman per year.

To tidy the data, we need to have an id, group, year and the measurement for calcium. We will have to read the CSV, add an ID column, rename the columns so they are easier to work with.

Our ultimate goal for the visualization is to compare the calcium levels over time between the two gropus and we can use a line plot for that.

CoPilot's informed prompt and response:

Prompt: I have a CSV file called calcium.csv that contains longitudinal calcium measurements for 31 women over four years. The first four columns (y0–y3) belong to the null treatment group, and the next four columns (y0–y3 again) belong to the diet/exercise treatment group. Each row is one woman. The data are not tidy because time is spread across columns and the treatment group is encoded by column position instead of an actual variable. Make a plan that would tidy this data and then create a plot that compares calcium levels over time between the two treatment groups

Response:

Plan for Tidying the Calcium Data and Making a Plot To start, the calcium.csv file isn't tidy because the time points (y0–y3) are spread across columns, and the treatment groups are basically "hidden" in the column positions. So the first thing I need to do is reshape the data so each row represents one woman at one time point.

1. Tidy the data Read the CSV exactly as it is.

Add an ID column (1–31) so each woman has a unique identifier.

Rename the columns so it's clear which ones belong to the null group and which belong to the diet/exercise group.

Pivot the data longer so that all the y0–y3 columns become a single year variable and a single calcium variable.

Create a new group variable based on whether the measurement came from the first four columns (null) or the last four (diet/exercise).

Convert the year labels ("y0", "y1", etc.) into numeric values 0–3.

Check for missing values since the last row of the CSV has blanks.

The final tidy dataset should have: id, group, year, calcium

2. Plan the visualization The goal is to compare how calcium levels change over time between the two groups. A line-based visualization makes the most sense.

I'd make:

A spaghetti plot where each woman is a faint line (to show individual variation)

On top of that, add bold mean lines for each group so the overall trends are easy to see

x-axis = year

y-axis = calcium

color = treatment group

This should make it obvious whether the diet/exercise group maintains calcium levels better than the null group.

3. Steps to make the plot in R Use the tidy dataset.

ggplot with aes(x = year, y = calcium, group = id, color = group)

Add geom_line(alpha = 0.3) for individual women.

Add stat_summary(fun = mean, geom = "line", linewidth = 1.2) for group averages.

Use a clean theme like theme_bw().

Label axes clearly and give the plot a descriptive title.

4. Expected insights Once the plot is made, I should be able to see:

Whether the diet/exercise group declines more slowly

Whether the null group shows a sharper drop over the four years

How much variation there is within each group

Whether the intervention seems to have a noticeable effect by year 3

##Generic Response

Prompt: Create a data visualization for these data. Calcium.csv

I've created a clear data visualization based on your calcium.csv file. Visualization Created: Boxplot of Calcium Measurements The figure below is a box-and-whisker plot showing the distribution of calcium values for each measurement series (y0, y1, y2, y3, repeated across groups). Why a boxplot? A boxplot is well suited for this dataset because it: • Shows central tendency (median) • Displays spread and variability (quartiles and range) • Highlights outliers • Allows easy comparison across multiple groups What you can see • Differences in median calcium levels across measurement series • Variation within each series • Potential outliers or unusually low/high measurements The visualization

If you'd like, I can also: • Create line plots (e.g., trends across rows) • Aggregate repeated variables (y0–y3) and visualize means ± SD • Produce publication-ready figures (journal style, grayscale, higher resolution) • Export the visualization to PNG, PDF, or SVG Just tell me how you plan to use it (exploratory analysis, presentation, or publication).

Prompt: What is your current version information?

Here's my current version information: • Product: Microsoft 365 Copilot (M365 Copilot) • Underlying model: GPT 5 chat model • Knowledge cutoff: 2025 • Current date/time

(for this response): March 31, 2026 (GMT 04:00) Practically speaking, that means: • I can reason, analyze data, write code, and create visualizations. • I can also look up up to date information when needed using web tools. • My built in training knowledge goes up through 2025, with newer facts retrieved on demand. If you're asking because you need to cite the tool in a methods section or documentation, I can help you phrase that appropriately.

(This chat took place on Tuesday, March 31st, 2026.)

Comparing and Contrasting

The plan informed Gen-AI response compared to the generic prompt response was very significant. The one that had a much more detailed prompt about the plan immediately recognized that the calcium data had columns that represented time points and that the two treatment groups were incorporated in the column positions. This felt very similar to the plan that I personally created where I reshaped the data into a much better and long format to make a plot that would highlight the trends across the years.

The generic prompt response was much less detailed was very vague compared to the informed prompt response. Due to the fact that the prompt just told the tool to create a visualization based on these data, the AI tool just treated the dataset as if it was a set of unrelated columns and gave a boxplot comparing the distributions of the years.

What I learned after comparing the two prompts and their respective responses was that it is extremely important to ensure that you give the GenAI tools very detailed prompts and context. This will allow the tool to produce something very accurate to what you have pictured in your mind and intended compared to a simple visualization that barely matched the scientific question at all.

Self Reflection

Overall this semester, I feel like I have a gotten a very good functional understanding of the programming language R, which is a language I had not used or learned about before. Additionally, throughout this semester, I have learned the fundamental aspects to being a good statistician along with a being a good programmer. For example in one of our first assignments, we had to write down the steps to make a paper boat to exactly replicate how someone else made the boat. This assignment taught me how you can be satisfied with your written steps and you can always add more details to give as much context to the readers as possible. Ultimately, this course has taught me the basics on how to use R to analyze data in a way which that is easy to understand and efficiently provide the message to the users. This was portrayed in our assignment where we had to tidy and wrangle the data first before making tables and plots relevant to the context at hand.

GenAI Usage Appendix

Tool used:CoPilot Date of usage: 04/15/2026 Prompt: I have a CSV file called calcium.csv that contains longitudinal calcium measurements for 31 women over four years. The first four columns (y0–y3) belong to the null treatment group, and the next four columns (y0–y3 again) belong to the diet/exercise treatment group. Each row is one woman. The data are not tidy because time is spread across columns and the treatment group is encoded by column position instead of an actual variable. Make a plan that would tidy this data and then create a plot that compares calcium levels over time between the two treatment groups

Response by the tool: Plan for Tidying the Calcium Data and Making a Plot To start, the calcium.csv file isn't tidy because the time points (y0–y3) are spread across columns, and the treatment groups are basically "hidden" in the column positions. So the first thing I need to do is reshape the data so each row represents one woman at one time point.

1. Tidy the data Read the CSV exactly as it is.

Add an ID column (1–31) so each woman has a unique identifier.

Rename the columns so it's clear which ones belong to the null group and which belong to the diet/exercise group.

Pivot the data longer so that all the y0–y3 columns become a single year variable and a single calcium variable.

Create a new group variable based on whether the measurement came from the first four columns (null) or the last four (diet/exercise).

Convert the year labels ("y0", "y1", etc.) into numeric values 0–3.

Check for missing values since the last row of the CSV has blanks.

The final tidy dataset should have: id, group, year, calcium

2. Plan the visualization The goal is to compare how calcium levels change over time between the two groups. A line-based visualization makes the most sense.

I'd make:

A spaghetti plot where each woman is a faint line (to show individual variation)

On top of that, add bold mean lines for each group so the overall trends are easy to see

x-axis = year

y-axis = calcium

color = treatment group

This should make it obvious whether the diet/exercise group maintains calcium levels better than the null group.

3. Steps to make the plot in R Use the tidy dataset.

ggplot with aes(x = year, y = calcium, group = id, color = group)

Add geom_line(alpha = 0.3) for individual women.

Add stat_summary(fun = mean, geom = "line", linewidth = 1.2) for group averages.

Use a clean theme like theme_bw().

Label axes clearly and give the plot a descriptive title.

4. Expected insights Once the plot is made, I should be able to see:

Whether the diet/exercise group declines more slowly

Whether the null group shows a sharper drop over the four years

How much variation there is within each group

Whether the intervention seems to have a noticeable effect by year 3

{{< pagebreak >}}
Code Appendix

```
library(dplyr)
library(tidyr)
library(knitr)
library(kableExtra)

airports <- data.frame(
  Airport = rep(c("ATL", "FRA", "PKX", "DXB", "LAX", "LHR"), each = 6),
  Year = rep(2020:2025, times = 6),
  Passengers = c(
    # ATL
    42.9, 75.7, 91.2, 100.1, 104.5, 107.0,
    # FRA
    29.3, 46.2, 60.0, 65.5, 68.0, 70.3,
    # PKX
    5.0, 20.0, 45.0, 60.0, 72.0, 85.0,
    # DXB
    25.0, 55.0, 70.0, 75.0, 80.0, 85.0,
    # LAX
    30.0, 50.0, 65.0, 70.0, 75.0, 78.0,
```

```r
    # LHR
    28.0, 48.0, 62.0, 67.0, 70.0, 73.0
  )
)

airports <- airports %>%
  mutate(
    Airport = as.factor(Airport),
    Year = as.integer(Year),
    Passengers = as.numeric(Passengers)
  )

head(airports)

options(knitr.kable.NA = "")

airports_table <- airports %>%
  arrange(Airport, Year) %>%
  pivot_wider(
    names_from = Year,
    values_from = Passengers
  )

airports_table %>%
  kable(
    caption = "Table 1. Passenger Traffic (2020-2025) for Six Major Airports",
    digits = 1,
    align = "lcccccc",
    format.args = list(big.mark = ",")
  )


library(dplyr)
library(tidyr)
library(knitr)
library(kableExtra)
library(ggplot2)

airports <- data.frame(
  Airport = rep(c("ATL", "FRA", "PKX", "DXB", "LAX", "LHR"), each = 6),
  Year = rep(2020:2025, times = 6),
  Passengers = c(
```

```r
    # ATL
    42.9, 75.7, 91.2, 100.1, 104.5, 107.0,
    # FRA
    29.3, 46.2, 60.0, 65.5, 68.0, 70.3,
    # PKX
    5.0, 20.0, 45.0, 60.0, 72.0, 85.0,
    # DXB
    25.0, 55.0, 70.0, 75.0, 80.0, 85.0,
    # LAX
    30.0, 50.0, 65.0, 70.0, 75.0, 78.0,
    # LHR
    28.0, 48.0, 62.0, 67.0, 70.0, 73.0
  )
)

airports <- airports %>%
  mutate(
    Airport = as.factor(Airport),
    Year = as.integer(Year),
    Passengers = as.numeric(Passengers)
  )

head(airports)

ggplot(airports, aes(x = Year, y = Passengers, color = Airport, group = Airport)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Figure 1: Passenger Traffic Trends for Six of the Busiest Airports (2020-2025)"
    x = "Year",
    y = "Passengers (in millions)",
    color = "Airport Code"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    plot.title = element_text(hjust = 0.5)
  )
generate_mc_points <- function(n, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n, x_min, x_max),
    y = runif(n, y_min, y_max)
```

```r
  )
}

library(tidyverse)
library(ggplot2)

make_mc_plot <- function(n, x_min = -4, x_max = 4, y_min = 0, y_max = 0.4) {

  pts <- generate_mc_points(n, x_min, x_max, y_min, y_max) %>%
    mutate(
      f_x = dnorm(x, mean = 0, sd = 1),
      flag = if_else(y <= f_x, "on/below", "above")
    )

  rect_area <- (x_max - x_min) * (y_max - y_min)
  p_hat <- mean(pts$flag == "on/below")
  est_int <- p_hat * rect_area

  ggplot(pts, aes(x = x, y = y, color = flag)) +
    stat_function(
      fun = dnorm,
      args = list(mean = 0, sd = 1),
      xlim = c(x_min, x_max),
      linewidth = 1,
      color = "black"
    ) +
    geom_point(alpha = 0.6, size = 1.3) +
    scale_color_manual(values = c("above" = "red", "on/below" = "blue")) +
    labs(
      title = paste("Monte Carlo Integration of Standard Normal PDF"),
      subtitle = paste("Resolution n =", n),
      caption = paste("Estimated Integral:", round(est_int, 4)),
      x = "x",
      y = "y",
      color = "Point Type"
    ) +
    theme_bw() +
    theme(
      legend.position = "none",
      plot.title = element_text(size = 12, face = "bold"),
      plot.subtitle = element_text(size = 10)
    )
```

```r
}

set.seed(184)

plot10     <- make_mc_plot(10)
plot100    <- make_mc_plot(100)
plot1000   <- make_mc_plot(1000)
plot10000  <- make_mc_plot(10000)

plot10
plot100
plot1000
plot10000
```